TD2 — Setup check, Git step by step, branches

Git, Python Environments & Coding Best Practices

MSA-DATB07-03 — 1 h 30 practical — **answer key**

Learning outcome. Check that your tools work; run the Git loop (status, add, commit, log, diff) on your own project and undo with restore and revert; create branches, merge them into `main` (fast-forward and three-way) and resolve a merge conflict.

Tip — how this TD works

As in TD1: **guided** steps give the command (predict, run, explain), then **Your turn** questions ask you to adapt a command from the exercise or from the slides. Questions marked ★ are **key questions**, the kind of reasoning asked in the quiz: write the command you typed and your answer in `td2_answers.md`, in your `courses/git` folder. This TD is not graded. Everybody must reach Part C: do not stay stuck more than 5 minutes, ask your neighbour or the instructor. The bonus is for those who finish early.

Part 0 — Check your setup *(15 min)*

Exercise 1 — Five checks

Open VS Code, then a terminal inside it: menu *Terminal* → *New Terminal* (or `Ctrl+ù` on a French keyboard, `Ctrl+`` on an English one). Run each command and compare with the expected answer. If it differs, apply the fix, then run the command again.

command	expected	if not
`echo $0`	`bash` or `/usr/bin/bash` (Windows), `zsh` or `-zsh` (macOS)	an empty line: the terminal is PowerShell. `Ctrl+Shift+P`, type *default profile*, choose *Terminal: Select Default Profile* → *Git Bash*; close the terminal (trash icon) and open a new one.
`git --version`	`git version 2...`	Git is not installed: TD1, Part 0.
`python --version` (macOS: `python3 --version`)	`Python 3.10` or newer	see the box below.
`code --version`	a version number	macOS: in VS Code, `Cmd+Shift+P`, *Shell Command: Install 'code' command in PATH*, then a new terminal.
`git config --global --list`	`user.name=...` `user.email=...` `init.defaultbranch=main` `core.editor=code --wait`	a line is missing: run the matching `git config --global` command of TD1, exercise 2.

Pitfall — Windows: "python: command not found", or the Microsoft Store opens

Two possible causes. Check first with `py --version`: if it prints a version, Python is installed but not on the `PATH` (the list of folders where the terminal looks for programs).

1. **Python is not on the PATH.** Run the Python installer again (the file you downloaded), choose *Modify*, *Next*, tick *Add Python to environment variables*, *Install*.
2. **Windows sends `python` to the Store.** *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*: turn off *python.exe* and *python3.exe*.

Then **quit VS Code completely and open it again**: a terminal reads the `PATH` only when it starts.

Pitfall — "code: command not found", and the fix above did not work

You can do the whole TD without the `code` command:

- **To open a file**: in VS Code, *File* → *Open Folder*, choose your `courses/git/td2` folder once; then click the file in the *Explorer* panel on the left. Each time the TD says `code file`, open the file this way.
- **To read a file**: `cat file` in the terminal.
- **When Git needs a message** (`git revert`, a merge): Git cannot open VS Code either. Tell Git to use *nano*, a small editor inside the terminal: `git config --global core.editor nano`. In nano: the message is ready, `Ctrl+O` `Enter` saves, `Ctrl+X` quits.

Guided. Last setup step: your course folder, and the `git lg` shortcut used all along this TD.

Your course folder can be anywhere you like: in *Documents*, on the *Desktop*, in a *dev* folder… This TD uses `~/Documents/courses/git` as the example. Two rules: end it with `courses/git`, so that the boxes "Where you should be" still match the end of your `pwd`; and no spaces in the folder names you create. If you chose another place, replace `~/Documents` by it in every `cd` command of the TD.

```
mkdir -p ~/Documents/courses/git/td2   # -p: the whole chain, no error if it exists
cd ~/Documents/courses/git/td2         # or: "cd ", drag the td2 folder, Enter
pwd                                    # everything of TD2 goes here
git config --global alias.lg "log --oneline --graph --all"   # the git lg command
```

Then open the `td2` folder in VS Code: *File* → *Open Folder*, choose `td2` (or type `code .` in the terminal: `.` = the current folder). VS Code reloads: the *Explorer* panel on the left shows `td2`, and a new terminal opens directly in `td2`. Create your answers file `td2_answers.md` there (box below). Everything of today goes into `td2`: your answers, your own project (Part A), the repository of Part B. Your course folder then has one folder per TD:

```
courses/git/
|-- td1_answers.md, td1-shell-and-git/      (TD1)
`-- td2/
    |-- td2_answers.md
    |-- my_budget/                    (Part A)
    `-- td2-branches-and-merges/      (Part B)
```

Instructor note

Most frequent problems: the VS Code terminal is still PowerShell (`echo $0` prints an empty line); `python` not found on Windows (PATH not ticked, or the Store alias); VS Code not restarted after a fix. If a laptop cannot be fixed in 10 minutes, pair the student with a neighbour for Part A and fix it during Part B.

Tip — create or edit a file: two methods, same result

Method 1 — in VS Code (the one used in this TD)

- **Create**: in the *Explorer* panel, right-click the folder → *New File*, type the name, `Enter`.
- **Edit**: click the file to open it, type. Press `Enter` at the end of each line.
- **Save**: `Ctrl+S` (macOS: `Cmd+S`). A white dot on the tab = not saved yet: Git does not see your edit until you save.

Method 2 — in the terminal (if you are stuck, or faster once you know it)

- `echo "rent: 600" > budget.txt`: creates the file with this line (or *replaces* its whole content).
- `echo "food: 250" >> budget.txt`: adds a line at the end.
- `code budget.txt`: opens the file in VS Code.

Git only sees the file on the disk: it makes no difference which method wrote it.

Part A — Git step by step, on your own project *(25 min)*

Exercise 2 — The loop

Where you should be: `pwd` ends with `courses/git/td2`
If not: `mkdir -p ~/Documents/courses/git/td2 && cd ~/Documents/courses/git/td2`

Guided. A tiny project: a budget, one line per expense. Predict, run, explain:

```
mkdir my_budget && cd my_budget      # && = then, if the first command worked
git init
```

In VS Code, create the file `budget.txt` inside `my_budget`, type the line `rent: 600`, press Enter, save. (Method 2: `echo "rent: 600" > budget.txt`.) Then:

```
git status                           # (1)
git add budget.txt
git status                           # (2)
git commit -m "Add rent"             # (3)
git log --oneline
```

1. In (1), under which title is `budget.txt` listed? What does it mean?
2. In (2), under which title is it now?
3. In (3), which part of the output is the identifier of the commit?

Your turn ★ Add a second line `food: 250` at the end of `budget.txt` and save, look at the change with `git diff` before staging it, then make a second commit with the message `Add food`. Write your commands, and what `git log --oneline` shows now.

Answer

(1) *Untracked files*: the file is in the folder but Git does not follow it. (2) *Changes to be committed*, `new file: budget.txt`: it is staged. (3) `[main (root-commit) 1a2b3c4] Add rent`: `1a2b3c4` (7 characters, different for everyone).

```
# in VS Code: line 2 = food: 250, save (or: echo "food: 250" >> budget.txt)
git diff                             # +food: 250
git add budget.txt
git commit -m "Add food"
git log --oneline                    # 2 lines: Add food (HEAD -> main), Add rent
```

Common mistakes: the file not saved (`git diff` shows nothing, white dot on the tab); method 2 with `>` instead of `>>` (the file contains only `food: 250`, `git diff` shows `-rent: 600`: `git restore budget.txt`, then redo). If Enter was not pressed after `rent: 600`, `git diff` shows `-rent: 600`, `+rent: 600` and "No newline at end of file": harmless, Git only notes the missing end of line.

Exercise 3 — Undo

Where you should be: `pwd` ends with `courses/git/td2/my_budget`
If not: `cd ~/Documents/courses/git/td2/my_budget`

Guided. An edit that was never committed. In VS Code, replace the whole content of `budget.txt` by `oops`, and save. (Method 2: `echo "oops" > budget.txt`.) Then predict, run, explain:

```
cat budget.txt
git status
git restore budget.txt
cat budget.txt
```

1. What did `git restore` do? Look at the tab of `budget.txt` in VS Code too.
2. Could you get `oops` back now? Why?

Your turn ★ Add a line `holidays: 5000`, commit it with the message `Add holidays`. You regret it: cancel this commit with `git revert` (VS Code opens with a message: save, close the tab). Then check with `git log --oneline` and `cat budget.txt`. How many commits are there, and is `Add holidays` still in the history?
Your turn ★ Create a file `private.txt`, then make Git ignore it: create a `.gitignore` that contains the line `private.txt`, check with `git status`, and commit the `.gitignore`.

Answer

`git restore` brought back the version of the last commit (rent and food); the VS Code tab shows it too, VS Code reloads the file from the disk. `oops` is lost for good: it was never committed, Git never saw it.

```
# in VS Code: add the line holidays: 5000, save (or: echo "holidays: 5000" >>
    budget.txt)
git add budget.txt
git commit -m "Add holidays"
git revert HEAD                  # VS Code: Revert "Add holidays", save, close
git log --oneline                # 4 commits: Revert "Add holidays", Add holidays,
    Add food, Add rent
cat budget.txt                   # rent and food, no holidays
# in VS Code: new file private.txt (any text), new file .gitignore with the line
    private.txt, save both
#   (or: echo "my notes" > private.txt ; echo "private.txt" > .gitignore)
git status                       # only .gitignore is listed, not private.txt
git add .gitignore
git commit -m "Ignore private notes"
```

4 commits: `Add holidays` is still in the history; `revert` added a commit that does the opposite. `git revert <id>` with the identifier of `Add holidays` is also correct.

Part B — Branches and two merges *(25 min)*

Exercise 4 — Download the repository, read the graph

Where you should be: `pwd` ends with `courses/git/td2`
If not: `mkdir -p ~/Documents/courses/git/td2 && cd ~/Documents/courses/git/td2`

The project computes revenue figures from `data/sales.csv`. Three colleagues each prepared a branch: `feature/readme`, `feature/plot`, `feature/stats`. Your job today: bring their work into `main`.

Guided. Predict, run, explain:

```
git clone https://github.com/msa-datb07-git/td2-branches-and-merges.git
cd td2-branches-and-merges
git branch                       # (1) the branches on your laptop
git branch -a                    # (2) -a = all: also the branches that are on GitHub
git switch feature/readme        # (3) create your local copy of each branch
git switch feature/plot
git switch feature/stats
git switch main
git branch                       # (4)
git lg
```

1. In (1), how many branches do you have? In (2), which new lines appear, and what do they start with?
2. After (3), what does `git branch` show in (4)? What did the `git switch` commands create?
3. On which branch are you? How do you see it in `git branch`, and in `git lg`?

Lines starting with `remotes/origin/` (`origin/` in `git lg`) are the branches as they are on GitHub: session 3 explains them.

Your turn ★ One of the three branches starts from the *last* commit of `main`: merging it will be a fast-forward. Which one, and how do you see it on the graph?

Answer

```
$ git branch                          $ git branch -a
* main                                * main
                                        remotes/origin/HEAD -> origin/main
                                        remotes/origin/feature/plot
                                        remotes/origin/feature/readme
                                        remotes/origin/feature/stats
                                        remotes/origin/main
```

(1) One branch, `main`: `git clone` creates only the main branch on the laptop. (2) The branches of GitHub, starting with `remotes/origin/`. (3)–(4) `git branch` now lists `feature/plot`, `feature/readme`, `feature/stats` and `* main`: each `git switch` to a branch that existed only on GitHub created its local copy. You are on `main`: the `*` in `git branch`, `HEAD -> main` in `git lg`. `feature/readme` is the one to be fast-forwarded: on the graph its line starts right on top of `d37005f`, the last commit of `main`; `feature/plot` and `feature/stats` both start lower, at `a97a373` (tag `v0.1`), and `main` has moved since.

Exercise 5 — A fast-forward

Where you should be: `pwd` ends with `courses/git/td2/td2-branches-and-merges`
If not: `cd ~/Documents/courses/git/td2/td2-branches-and-merges`

Guided. Predict, run, explain:

```
git switch main
git merge feature/readme
git lg
```

1. Which word does Git print? Was a new commit created?
2. Where are the labels `main` and `feature/readme` now?

Answer

"Fast-forward". No new commit: `main` simply moved to `a634f54`, where `feature/readme` already was; both labels are on the same commit.

Exercise 6 — A three-way merge

Where you should be: `pwd` ends with `courses/git/td2/td2-branches-and-merges`
If not: `cd ~/Documents/courses/git/td2/td2-branches-and-merges`

Guided. Predict, run, explain:

```
git merge feature/plot      # VS Code opens with a message: keep it, save, close the tab
git lg
python plot.py              # the new script of feature/plot (macOS: python3)
```

1. What kind of commit was created? How do you see on the graph that two lines join?

Your turn ★ `main` and `feature/plot` both changed `analysis.py`, yet there was no conflict. Why? (Hint: `git show d37005f` and `git show 839e370` show what each commit changed; `q` to quit.)

Answer

```
git merge feature/plot      # "Merge made by the 'ort' strategy": a merge commit,
    two lines join on the graph
git show d37005f            # main changed total_revenue (middle of the file)
git show 839e370            # feature/plot added revenue_by_region further down
```

No conflict because the two sides changed different places of `analysis.py`, far from each other.

Part C — The conflict *(20 min)*

Exercise 7 — Provoke, read, cancel

Where you should be: `pwd` ends with `courses/git/td2/td2-branches-and-merges`
If not: `cd ~/Documents/courses/git/td2/td2-branches-and-merges`

Guided. Predict, run, explain:

```
git merge feature/stats
git status
code analysis.py           # or click it in the Explorer; look at the conflict, change
    nothing yet
python analysis.py
git merge --abort
git status
```

1. Which file is in conflict, and under which label does `git status` list it?
2. In `analysis.py`, what does the `HEAD` side want, and what does the `feature/stats` side want?
3. Why does `python analysis.py` fail?
4. What did `git merge --abort` do?

Answer

`analysis.py`, listed under "Unmerged paths: both modified". The `HEAD` side (`main`) rounds `total_revenue` to the cent; the `feature/stats` side adds a `region=None` parameter that filters the rows by region. Python fails with a `SyntaxError` on the line `<<<<<<< HEAD`. `git merge --abort` cancels the merge: the folder is back exactly as before.

Exercise 8 — Resolve for real

Where you should be: `pwd` ends with `courses/git/td2/td2-branches-and-merges`
If not: `cd ~/Documents/courses/git/td2/td2-branches-and-merges`

Your turn ★ Merge `feature/stats` again and resolve the conflict so that *both* intentions survive: `total_revenue` must accept a `region` *and* round the result to the cent. Follow the slide "Step 13 — Resolve, add, commit". Before committing, check that:

```
grep -n "<<<<<<<" analysis.py       # must print nothing: no marker left
python analysis.py                  # must run
```

Then write in your answers file the final `total_revenue` function and the commands you used.
Your turn ★ Run `git lg`. Find on the graph the fast-forward of `feature/readme` and the two merge commits. How many commits does `main` have now? (`git log --oneline main | wc -l`)

Answer

Expected resolution:

```
def total_revenue(rows, region=None):
    """Sum quantity * unit_price, optionally restricted to one region, rounded to
    the cent."""
    if region is not None:
        rows = [r for r in rows if r["region"] == region]
    if not rows:
        return 0.0
    return round(sum(float(r["quantity"]) * float(r["unit_price"]) for r in rows),
    2)
```

```
git merge feature/stats
code analysis.py                  # keep both: parameter + filter + round(..., 2);
    delete the 3 marker lines
grep -n "<<<<<<<" analysis.py     # nothing
python analysis.py                # Total revenue: 6945.50 ...
git add analysis.py
git commit                        # VS Code: keep "Merge branch 'feature/stats'",
    save, close
```

Common errors: markers committed (`SyntaxError`), one side lost (check the docstring), commit refused because `git add` was forgotten. `main` has 10 commits. Expected graph (the two merge identifiers differ per student; `a634f54` is the fast-forward, nothing was created there):

```
*   ee8145f (HEAD -> main) Merge branch 'feature/stats'
|\
| * e6adc17 (origin/feature/stats, feature/stats) Add region filter to
    total_revenue and average_order_value
* |   79c879d Merge branch 'feature/plot'
|\ \
| * | 8d60f59 (origin/feature/plot, feature/plot) Add a title to the bar chart
| * | 839e370 Add revenue_by_region and a text bar chart
| |/
* | a634f54 (origin/feature/readme, feature/readme) Document how to run the
    analysis
* | d37005f (origin/main, origin/HEAD) Round total_revenue to the cent
|/
* a97a373 (tag: v0.1) Add command-line entry point
* 36dc8a8 Add monthly_revenue
* 2363066 Initial project: loader, total revenue, data
```

Part D — Before session 3: a GitHub account *(5 min)*

Session 3 puts your work on GitHub. If you do not have a GitHub account yet, create one on `https://github.com` **before session 3**:

1. Look at the e-mail Git writes on your commits: `git config --global user.email`.
2. Sign up on GitHub **with that e-mail**. If you already have an account with another e-mail, add this one: your avatar → *Settings* → *Emails* → *Add email address*.

Why the same e-mail: GitHub links a commit to your account through the e-mail written in it. With another e-mail, your commits appear under a grey, unknown name.

Bonus — If you finish early

The appendix of the session 2 slides explains these commands.

Exercise 9 — Rebase your own branch

Where you should be: `pwd` ends with `courses/git/td2/td2-branches-and-merges`
If not: `cd ~/Documents/courses/git/td2/td2-branches-and-merges`

Guided. A branch can start from any commit: the last word of `git switch -c` gives the starting point. Predict, run, explain:

```
git switch -c feature/tests v0.1      # a new branch that starts at the tag v0.1, not at
    main
code tests.py                         # or New File in the Explorer; write the 3 lines
   below, save
git add tests.py && git commit -m "Add a test for total_revenue"
git lg                                # note the identifier of your commit
git rebase main                       # replay your commit on top of main
git lg                                # look at the identifier again
```

```
import analysis
assert analysis.total_revenue([]) == 0.0
print("ok")
```

Your turn The identifier of your commit changed with the rebase. Why? Then bring `feature/tests` into `main`: fast-forward or three-way merge?

Answer

```
git switch main
git merge feature/tests       # Fast-forward: after the rebase, the branch starts
    at the end of main
python tests.py               # ok
```

The identifier changed because rebase creates a copy of the commit with a new parent (the last commit of `main`): the identifier depends on the parent. The rebase was allowed because the commit existed only on the student's laptop (golden rule).

Exercise 10 — Put work aside with stash

Where you should be: `pwd` ends with `courses/git/td2/td2-branches-and-merges`
If not: `cd ~/Documents/courses/git/td2/td2-branches-and-merges`

Guided. Predict, run, explain:

```
git branch feature/cli v0.1     # a branch at v0.1, without switching to it
code analysis.py                # or from the Explorer; on main: add print("hello") at
   the end, save, do NOT commit
git switch feature/cli          # what does Git answer?
git stash
git status                      # what changed?
git switch feature/cli
git switch main
git stash pop
git restore analysis.py         # throw the print away
```

Your turn Why did Git refuse the first `git switch`, and what did `git stash` and `git stash pop` do? Then delete the merged branches with `git branch -d` and check with `git branch`.

Answer

Git refused because `analysis.py` is different on `feature/cli` (it starts at `v0.1`): switching would have overwritten the edit not committed. `git stash` put the edit aside and cleaned the folder; `git stash pop` brought it back. `git branch -d feature/readme feature/plot feature/stats feature/tests feature/cli` removes the labels; the commits stay in the history.

Deliverable

Nothing to send: TD2 is not graded. Keep your `td2_answers.md`: the ★ questions are the kind of reasoning asked in the quiz. Before session 3: a GitHub account with the e-mail of your `git config`.